Running ten new open-weight models on T4 and A100: inference failure modes and fixes

PinoAI engineering note · ENG-002

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-09-30

Summary

We evaluated ten recent open-weight checkpoints, five bases and their instruct siblings (Gemma 4 12B and E4B, Qwen3.5-9B and 4B, Sailor2-8B), on two workloads: greedy generation on our 175-prompt Filipino test set (PH-EVAL), and log-likelihood multiple-choice scoring (MCF) on FilBench [1] through lighteval. The hosts were a free Kaggle 2× T4 notebook and a rented single A100 80GB on Modal. METHODS-001 documents the FilBench harness itself. This note covers the layer around it: loading the models, their chat templates, GPU memory, and the plumbing that moves outputs from a remote container to the scorer.

We hit eight failure modes. Each is described below with its symptom, root cause, fix, verification, and effect on reported results.

Table 1: The eight failure modes at a glance. "Silent" means the run exited normally and produced a plausible artefact.

§	failure mode	silent?	evidence in the run files	effect on reported results
2.1	stock transformers 5.0.0 cannot load the checkpoints	loud	8 of 10 fail on the stock image	none; pinned to 5.17.0
2.2	bf16-native Gemma 4 run in fp16 on T4	would be silent	0 empty answers in 875 FULL generations	none detected
2.3	Qwen3.5 template thinks by default, Gemma 4's does not	silent	20/20 pre-fix answers are English reasoning; 7 reach `</think>`	none after the fix
2.4	Gemma 4 12B-it leaks a `thought` marker	silent	11 of 175 answers; 2 missed by the first regex	2 items mis-scored until the fix (61/70 to 63/70)
2.5	full-vocabulary logits exhaust a T4; the trim window re-grows on mixed-length batches	loud	3 of 3 full FilBench runs OOM on GPU 0	full runs moved to an A100
2.6	refused push reads the previous version's output	silent	3 unguarded queue entries; byte-identical outputs for 2 models	none; guard fired 3x on rerun
2.7a	`:` in a file name becomes an NTFS alternate data stream	silent	4 folders with a 0-byte file hiding 57 streams	none; affected runs not used
2.7b	warm container leaks earlier models' files	silent	207 foreign details files in later folders	none; scorer filters by model

Silent failures are the dangerous kind in a model comparison: each one could have attached one model's numbers to another, or scored formatting instead of content. The runner now carries a guard for each (Section 3). The last gap found, two thought-marker variants the scorer's first regex missed (2.4), was fixed in the scorer while this note was being written; the published PH-EVAL table still carries the old value.

1. Setup

Models and pins. The ten checkpoints are pinned to full commit SHAs, resolved from `configs/models.yaml` through the Hugging Face API at launch and recorded in each run's `run_meta.json`. Three tables list every run behind the reported numbers, all read from the run files: identity (Table 2: host, revision, library version, dtype), settings (Table 3: loaded class or scorer, template, logit trimming, batch, items) and outcome (Table 4). In these tables "T4x2" is the

Kaggle machine shape `NvidiaTeslaT4` (two Tesla T4s in the FilBench kernels' `nvidia-smi` output; the device name the generation payload printed for PH-EVAL), and "A100" is the A100 80GB PCIe that Modal reported.

Hosts.

- **Kaggle, machine shape `NvidiaTeslaT4`**: two Tesla T4s. `nvidia-smi` reports 15,360 MiB each, and PyTorch reports a total capacity of 14.56 GiB per GPU. No bf16. The account allows two concurrent GPU sessions, and a third push is refused with "Maximum batch GPU session count of 2 reached". Models are split across both GPUs with `device_map="auto"` and a per-GPU `max_memory` cap.
- **Modal, one A100 80GB**: the device variant is assigned by the platform and cannot be selected. The harness recheck in METHODS-001 got an A100-SXM4-80GB. The three full FilBench runs got an A100 80GB PCIe, as recorded in their `run_meta.json`.

Software. transformers 5.17.0 everywhere, pinned explicitly (2.1). lighteval 0.13.1.dev0, installed from the git main branch and patched as described in METHODS-001. Generation is greedy (`do_sample=False`), fp16, at most 512 new tokens, with the chat template applied and `enable_thinking=False`. MCF runs use the chat template off, fp16 and batch size 4. `keep_last_logits` is on for the T4 runs and off on the A100.

Workloads.

- **PH-EVAL FULL** (instruct models): 175 prompts, one per forward, on Kaggle.
- **FilBench MCF** (bases): the governance-cleared, non-gated, non-script-loader MCF tasks. Global-MMLU is capped at 20 items per subject. Sample checks ran on the first 20 items per task on Kaggle; full runs used every item.

Table 2: Run identity: every run behind the reported numbers for the ten models, from `run_meta.json` and the kernel logs. Revisions are the first 8 characters of the pinned commit SHA.

model	workload	host	revision	transformers	dtype
gemma-4-12B-it	PH-EVAL	T4x2	`707f0a3b`	5.17.0	fp16
Qwen3.5-9B	PH-EVAL	T4x2	`c2022362`	5.17.0	fp16
Sailor2-8B-Chat	PH-EVAL	T4x2	`577d5b27`	5.17.0	fp16
Qwen3.5-4B	PH-EVAL	T4x2	`851bf6e8`	5.17.0	fp16
gemma-4-E4B-it	PH-EVAL	T4x2	`ee0ef602`	5.17.0	fp16
gemma-4-12B	FilBench	T4x2	`023679ed`	5.17.0	fp16
gemma-4-12B	FilBench	A100	`023679ed`	5.17.0	fp16
Qwen3.5-9B-Base	FilBench	T4x2	`68c46c4b`	5.17.0	fp16
Qwen3.5-9B-Base	FilBench	A100	`68c46c4b`	5.17.0	fp16
Sailor2-8B	FilBench	T4x2	`fb9e3b99`	5.17.0	fp16
Sailor2-8B	FilBench	A100	`fb9e3b99`	5.17.0	fp16
Qwen3.5-4B-Base	FilBench	T4x2	`1001bb4d`	5.17.0	fp16
gemma-4-E4B	FilBench	T4x2	`411aa17b`	5.17.0	fp16

Table 3: Run settings. For PH-EVAL the class is the one `from_pretrained` returned; for FilBench the scorer is lighteval. "keep last logits" is the logit-trimming wrapper of 2.5; "items" is items per task (FilBench) or prompts (PH-EVAL).

model	class / scorer	template	keep last logits	batch	items
gemma-4-12B-it	Gemma4Unified ForConditional Generation	thinking off	n/a	1	175
Qwen3.5-9B	Qwen3_5 ForCausalLM	thinking off	n/a	1	175
Sailor2-8B-Chat	Qwen2 ForCausalLM	thinking off	n/a	1	175
Qwen3.5-4B	Qwen3_5 ForCausalLM	thinking off	n/a	1	175
gemma-4-E4B-it	Gemma4 ForConditional Generation	thinking off	n/a	1	175
gemma-4-12B	lighteval 0.13.1.dev0	chat template off	on	4	all
gemma-4-12B	lighteval 0.13.1.dev0	chat template off	off	4	all
Qwen3.5-9B-Base	lighteval 0.13.1.dev0	chat template off	on	4	all
Qwen3.5-9B-Base	lighteval 0.13.1.dev0	chat template off	off	4	all
Sailor2-8B	lighteval 0.13.1.dev0	chat template off	on	4	all
Sailor2-8B	lighteval 0.13.1.dev0	chat template off	off	4	all
Qwen3.5-4B-Base	lighteval 0.13.1.dev0	chat template off	on	4	20/task
gemma-4-E4B	lighteval 0.13.1.dev0	chat template off	on	4	20/task

Table 4: Run outcome. "run_id ok" means the `run_id` in the downloaded `run_meta.json` equals the one the launcher generated and logged at push time. "files" counts lighteval details files.

model	workload	host	outcome
gemma-4-12B-it	PH-EVAL	T4x2	175/175, run_id ok
Qwen3.5-9B	PH-EVAL	T4x2	175/175, run_id ok
Sailor2-8B-Chat	PH-EVAL	T4x2	175/175, run_id ok
Qwen3.5-4B	PH-EVAL	T4x2	175/175, run_id ok
gemma-4-E4B-it	PH-EVAL	T4x2	175/175, run_id ok
gemma-4-12B	FilBench	T4x2	OOM, GPU 0
gemma-4-12B	FilBench	A100	complete (69 files)
Qwen3.5-9B-Base	FilBench	T4x2	OOM, GPU 0
Qwen3.5-9B-Base	FilBench	A100	complete (69 files)
Sailor2-8B	FilBench	T4x2	OOM, GPU 0
Sailor2-8B	FilBench	A100	complete (69 files)
Qwen3.5-4B-Base	FilBench	T4x2	complete (20/task)
gemma-4-E4B	FilBench	T4x2	complete (20/task)

2. Failure modes

2.1 Framework version gating

Symptom. On Kaggle's stock image, `AutoConfig`/`AutoModelForCausalLM` raised `ValueError: The checkpoint you are trying to load has model type 'qwen3_5' but Transformers does not recognize this architecture` for 8 of the 10 checkpoints. Only Sailor2 (a `qwen2` architecture) loaded.

Root cause. The stock image ships transformers 5.0.0, which predates the `qwen3_5`, `gemma4` and `gemma4_unified` model types. Installing lighteval does not upgrade it, because lighteval's requirement is satisfied by 5.0.0.

Fix. Pin `transformers==5.17.0` and install it *after* lighteval, so lighteval's resolver cannot leave the older version in place. The pin is a single constant shared by every launcher, and each run records the version it actually imported. A pin rather than `-U` keeps every baseline on the same version.

How verified. A CPU-only probe kernel (0 GPU-hours) loaded the config and resolved the auto class for all ten checkpoints at three stages: stock, after the lighteval install, and after the pin (Table 5). Every GPU run then recorded `transformers 5.17.0` (Table 2).

Table 5: Auto-class resolution per checkpoint on Kaggle's stock image, after installing lighteval, and after pinning transformers 5.17.0 (CPU-only probe; torch 2.10.0+cpu). "checkpoint dtype" is the `torch_dtype` in the checkpoint's config.

checkpoint	model type	stock (tf 5.0.0)	with lighteval (tf 5.0.0)	tf 5.17.0: class	dtype
gemma-4-12B	`gemma4_unified`	fails	fails	Gemma4Unified ForConditional Generation	bf16
gemma-4-12B-it	`gemma4_unified`	fails	fails	Gemma4Unified ForConditional Generation	bf16
Qwen3.5-9B-Base	`qwen3_5`	fails	fails	Qwen3_5 ForCausalLM	none
Qwen3.5-9B	`qwen3_5`	fails	fails	Qwen3_5 ForCausalLM	none
Sailor2-8B	`qwen2`	Qwen2 ForCausalLM	Qwen2 ForCausalLM	Qwen2 ForCausalLM	bf16
Sailor2-8B-Chat	`qwen2`	Qwen2 ForCausalLM	Qwen2 ForCausalLM	Qwen2 ForCausalLM	bf16
Qwen3.5-4B-Base	`qwen3_5`	fails	fails	Qwen3_5 ForCausalLM	none
Qwen3.5-4B	`qwen3_5`	fails	fails	Qwen3_5 ForCausalLM	none
gemma-4-E4B	`gemma4`	fails	fails	Gemma4 ForConditional Generation	bf16
gemma-4-E4B-it	`gemma4`	fails	fails	Gemma4 ForConditional Generation	bf16

Two class-level details matter downstream:

- Under 5.17.0, `AutoModelForCausalLM` maps Qwen3.5 to `Qwen3_5ForCausalLM`, the text-only model without the vision encoder. It maps both Gemma 4 sizes to their `*ForConditionalGeneration` class, which loads the full multimodal model.
- The weight-memory estimates in 2.5 use the checkpoints' total parameter counts. For Qwen3.5 those include the vision encoder, so they are upper bounds for the text-only class.

Effect on results. None. No result was produced before the pin.

2.2 fp16 inference of bf16-native Gemma 4 on T4

Symptom (risk). The Gemma 4 and Sailor2 configs declare bf16 (Table 5), and T4 (compute capability 7.5) has no bf16. Running a bf16-trained model in fp16 can overflow activations to `inf`/`NaN`. Under greedy decoding that shows up as empty or degenerate output, not as an exception.

Fix. All runs use fp16 (`torch_dtype=torch.float16`), recorded as `dtype: float16` in `run_meta.json`. The A100 runs also used fp16, even though the A100 supports bf16. This keeps the MCF runs on the dtype that the harness check in METHODS-001 validated, so T4 and A100 numbers do not differ by dtype.

How verified. On the FULL generations, 0 of 175 answers are empty for every model, including both Gemma 4 instruct models (Table 6, last column). The auto-scored accuracies of the Gemma models, 90.0% and 80.0% with the fixed scorer (Table 7), are not what a numerically broken model produces.

Effect on results. None detected. The check rules out catastrophic overflow only. It cannot detect a small fp16 versus bf16 accuracy shift, because no bf16 twin run exists (Section 5).

2.3 Chat-template thinking asymmetry

Symptom. In a 20-prompt sample run of Qwen3.5-4B, every answer opened with an English reasoning trace ("Thinking Process:", "Here's a thinking process…", "The user wants…"). Only 7 of 20 reached `</think>` within 512 new tokens (Table 6, first row). A scorer would have graded the trace, or a truncated trace, instead of an answer.

Root cause. The two families' templates default in opposite directions. Qwen3.5's template emits a thinking block unless `enable_thinking` is passed and false. Gemma 4's template leaves thinking off unless it is asked for. A harness that calls `apply_chat_template` without the flag therefore gives one family a reasoning budget and not the other, and at 512 tokens the reasoning budget is mostly spent on English.

Fix. Pass `enable_thinking=False` for every model. Templates without the variable ignore it. The flag is a launcher option, off by default, and is recorded in `run_meta.json`. The comparison is of answers, not reasoning traces, and the team agreed the choice before the FULL runs.

How verified. In the FULL runs, 0 of 175 answers open with a reasoning trace or contain `</think>`, for all five instruct models, and all five `run_meta.json` files record `enable_thinking: false` (Table 6).

Table 6: Reasoning traces per generation run, from `predictions.jsonl` (raw model text). "enable_thinking" is the value in `run_meta.json` ("not set" = template default). "reasoning trace" counts answers that open with one of the phrases seen in the pre-fix run; "empty" counts empty answers.

run	enable_thinking	answers	reasoning trace	</think>	empty
Qwen3.5-4B, sample, before fix	not set	20	20	7	0
gemma-4-12B-it, FULL	False	175	0	0	0
Qwen3.5-9B, FULL	False	175	0	0	0
Sailor2-8B-Chat, FULL	False	175	0	0	0
Qwen3.5-4B, FULL	False	175	0	0	0
gemma-4-E4B-it, FULL	False	175	0	0	0

Effect on results. None on reported numbers, since the fix came before the FULL runs. Without it, the Qwen3.5 models would have been scored on truncated English reasoning.

2.4 Thought-channel leak in Gemma 4 12B-it

Symptom. With thinking off, Gemma 4 12B-it still began some answers with the word `thought` on its own line, followed by the real answer, e.g. `thought\n58` for a numeric item.

Root cause. Gemma 4 uses a thought channel delimited by special tokens. With thinking disabled the model sometimes still opens the channel. Decoding with `skip_special_tokens=True` removes the delimiter tokens but keeps the ordinary word `thought` and the newline, so the leak reaches the answer text as plain text.

Fix. Keep the raw text in `predictions.jsonl` and strip the marker at load time in the scorer, for every model alike, before scoring and before judging. The first regex was `\A\s*\.?thought[ \t]*\n`: a leading `thought` line, optionally preceded by a period.

How verified. A unit test checks that the marker is stripped at the start only, and that words such as "thoughtful" are left alone. On the FULL run, the first regex removed the marker from 9 of 175 answers.

Second gap (found while writing this note, now fixed). Re-reading the raw predictions showed four marker variants: `thought`, `.thought`, `/thought` and `-thought`. The first regex covered only the first two. The two answers with `/thought` and `-thought` are both auto-scored multiple-choice items, and both are correct once the prefix is removed. The scorer's regex is now `\A\s*[./-]?thought[ \t]*\n`, the unit test covers the new cases, and the run's `scorecard.json` was regenerated. Gemma 4 12B-it's auto-scored result moves from 61/70 (87.1%) to 63/70 (90.0%). No other model is affected, and no answer in any run still starts with a `thought` line behind any punctuation character (Table 7; the generator script asserts both).

Table 7: Thought markers and the PH-EVAL auto-scored subset under the first and the fixed scorer regex, recomputed with the project's scorer; the fixed column equals each run's regenerated `scorecard.json`.

model	stripped, first regex	missed, first regex	score, first regex	score, fixed scorer	changed
gemma-4-12B-it	9	2	61/70 (87.1%)	63/70 (90.0%)	2
Qwen3.5-9B	0	0	54/70 (77.1%)	54/70 (77.1%)	0
Sailor2-8B-Chat	0	0	43/70 (61.4%)	43/70 (61.4%)	0
Qwen3.5-4B	0	0	48/70 (68.6%)	48/70 (68.6%)	0
gemma-4-E4B-it	0	0	56/70 (80.0%)	56/70 (80.0%)	0

Effect on results. The PH-EVAL auto-scored accuracy of Gemma 4 12B-it published in EVAL-001 (87.1%) was understated by 2 items; the corrected value is 90.0%. The model already ranked first on that subset, so the ranking does not change. EVAL-001 still has to be updated to the new value (Section 5).

2.5 MCF scoring memory: full-vocabulary logits on a 14.56 GiB GPU

Symptom. The first FilBench runs on 2× T4 ran out of memory on GPU 0 at the first log-likelihood batch, already at batch size 1 ("Tried to allocate 1.57 GiB", Sailor2-8B; METHODS-001, 3.3). Two things followed:

- The first fix passed sample runs. The full runs at batch 4 then failed again on GPU 0, with requests of 5.25 GiB (Sailor2-8B) and 10.09 GiB (Qwen3.5-9B-Base).
- Gemma 4 12B additionally failed on GPU 1 under the default weight split.

Root cause, in four parts.

1. **lighteval materialises `[rows, seq_len, V]` logits.** It then takes a `log_softmax` over them, although MCF only reads each choice's continuation positions. With `device_map="auto"` the output lands on GPU 0, on top of that GPU's share of the weights.
2. **"Batch size 1" still means one forward over all of a doc's choices.** lighteval's own warning says batch 1 "should be understood as '1 times the maximum number of choices per sample'". For a doc with 7 choices (the case estimated in METHODS-001, 3.3) × 1,000 tokens, the fp16 logits plus an fp32 `log_softmax` copy come to 5.9, 9.7 and 10.3 GiB for the 152K, 248K and 262K vocabularies (Table 8, Figure 1 a). That is 40–70% of a T4 before any weights.
3. **The first fix keeps one window per batch.** The wrapper computes logits only for the last `keep = seq_len − min(real_len) + 32` positions, via `logits_to_keep`. With right padding, one short doc in a batch of long ones (for example short items next to long readability passages) makes `keep` nearly `seq_len`. That brings back full-sequence logits.
   - Assuming fp16 `[rows, keep, V]` logits, the observed requests correspond to about 18.6K and 21.8K kept row-positions (Table 10).
   - For batch 4 × up to 7 choices, that is roughly 660–780 positions per row: essentially the whole prompt.
4. **The default weight cap starves GPU 1 for the 12B.** At 22.3 GiB of fp16 weights, the default 8 GiB cap on GPU 0 leaves about 14.3 GiB for GPU 1 (Table 9), which is nearly the whole 14.56 GiB device. The team's run notes record the sample run failing on GPU 1 with 14.43 GiB allocated. That kernel log was later overwritten by a rerun in the same folder, so this number is not re-derivable from files.
   - A balanced 11/12 GiB split fixed the load.
   - The full run at batch 4 then failed on GPU 0 with only a 0.61 GiB request, because 13.77 GiB was already allocated there (Table 10).

Table 8: Arithmetic estimate, not a measurement. Logit memory for one forward over 7 choices × 1,000 tokens: V × 7,000 × 2 bytes (fp16 logits) plus × 4 bytes (fp32 `log_softmax`). Vocabulary sizes are the checkpoints' `config.vocab_size`. Values in GiB.

model	vocabulary	fp16 logits (GiB)	fp32 log-softmax (GiB)	total (GiB)
Sailor2-8B	151,936	1.98	3.96	5.9
Qwen3.5-9B-Base	248,320	3.24	6.48	9.7
gemma-4-12B	262,144	3.42	6.84	10.3

Table 9: Weight placement on 2× T4, arithmetic. Weights are total parameters × 2 bytes (for Qwen3.5 this includes the vision encoder, which the text-only class does not load). The GPU 1 columns assume GPU 0 is filled to its cap and the rest of the weights go to GPU 1. Values in GiB.

model	fp16 weights (GiB)	on GPU 1, GPU 0 capped at 8	on GPU 1, 11/12 split
Sailor2-8B	15.9	7.9	4.9
Qwen3.5-9B-Base	18.0	10.0	7.0
gemma-4-12B	22.3	14.3	11.3

Table 10: The three out-of-memory failures of the full FilBench runs on Kaggle (batch 4, `keep_last_logits` on, all on GPU 0), parsed from the kernel logs. "Allocated" is memory already held by PyTorch on GPU 0. "Implied row-positions" = requested bytes / (2 × V), i.e. the number of (row, position) logit vectors an fp16 `[rows, keep, V]` tensor of that size holds.

model	weight cap GPU 0 / 1 (GiB)	requested (GiB)	free (GiB)	allocated (GiB)	implied row-positions
Sailor2-8B	8 / 14.5	5.25	1.38	12.95	18,551
Qwen3.5-9B-Base	8 / 14.5	10.09	5.30	9.01	21,815
gemma-4-12B	11 / 12.0	0.61	0.41	13.77	1,244

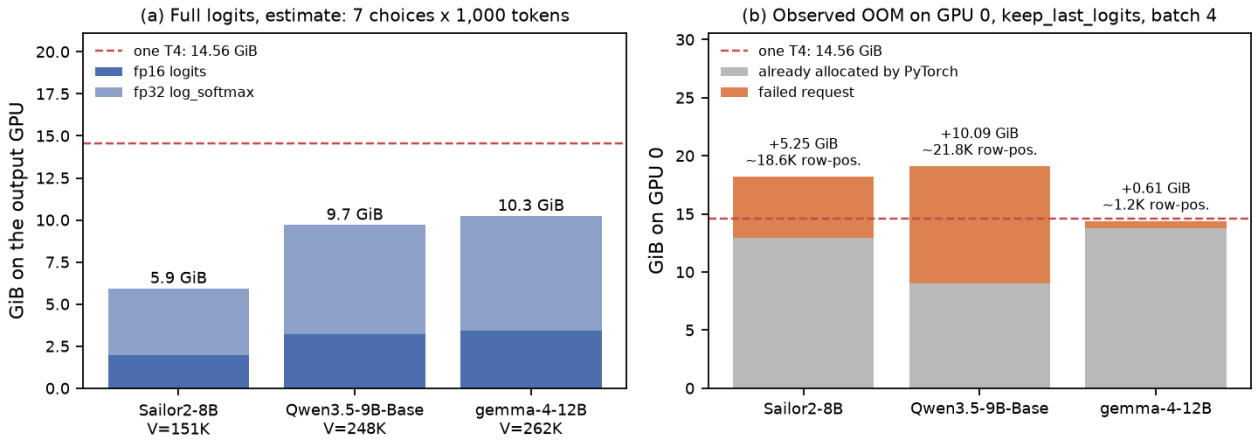


Figure 1: Left: estimated full-logit memory for one 7-choice, 1,000-token forward, per vocabulary. Right: the three out-of-memory events of the full FilBench runs on 2× T4 with `keep_last_logits` at batch 4, split into memory already held by PyTorch and the failed request. Dashed line: one T4's total capacity.

Fix. There are three layers, applied in order:

1. **Logit trimming.** A proxy around the Hugging Face model passes `logits_to_keep` and returns a `ShiftedLogits` object that lighteval indexes by the original absolute positions, so lighteval's code is unchanged. If lighteval ever asks for a position that was not computed, the proxy raises `IndexError` rather than returning a wrong value. `log_softmax` runs on the kept slice only.
2. **Weight split.** A per-GPU `max_memory` cap. GPU 0 is capped at 8 GiB by default to leave room for logits, and Gemma 4 12B uses 11/12 GiB.
3. **Batch shape or platform.**
   - On T4, the full-run driver now uses batch 1, so each forward holds one doc's choices, which have similar lengths.
   - On an A100 80GB, full logits fit, so the full runs moved there with `keep_last_logits` off.
   - A per-row index set (the union of each row's last positions, which `logits_to_keep` also accepts) would fix the wrapper itself. It has not been built.

How verified. The equivalence of trimmed and full logits was checked twice:

- On CPU, a unit test mirrors lighteval's scoring loop and gets identical per-choice log-probability sums with and without the wrapper.
- On GPU, an A/B arm with the wrapper off reproduced the failing harness run digit for digit (METHODS-001, 3.1 and 3.3).

The A100 runs themselves completed with the wrapper off (Table 3, Table 4).

Effect on results. None on scores. Scoring is deterministic, and the wrapper was shown to be score-neutral. It changed *where* the full FilBench runs could execute: all three bases ran on the A100. The T4 batch-1 path has not been exercised on a full run (Section 5).

2.6 Stale-output mislabelling

Symptom. In one sample queue, two FilBench runs reported exit code 0 within 1.7 and 0.1 minutes and wrote no results. A third run was refused outright. The two downloaded kernel logs are byte-identical, although they were fetched for two different models (Table 11).

Root cause. There were two defects in the Kaggle launcher, compounded by the session limit:

- **A refused push was only printed.** For Gemma 4 E4B the push was refused ("Maximum batch GPU session count of 2 reached"), but the launcher went on polling the notebook's status.
- **Status and output belong to the notebook, not to a run.** Every FilBench model is a new version of one fixed notebook (Kaggle attaches secrets per notebook). The first status read therefore returned the *previous* version's `COMPLETE`, and `kernels output` downloaded the previous version's files.
  - For Gemma 4 E4B this followed the refused push.
  - For Qwen3.5-4B-Base the push was accepted, but the status was read before the new version replaced the old one.

Fix.

- A refused push raises immediately ("push refused, nothing ran").
- Every push gets a unique `run_id`. The kernel writes it into `run_meta.json` as its first action, and the launcher refuses any download whose `run_meta.json` carries a different `run_id`.
- Old outputs are deleted before download.
- The first status read waits 30 s.
- The queue drivers stop the whole queue when a run fails in under a minute, instead of cascading into the next model.
- Concurrent use of the two GPU sessions by different people is coordinated by an explicit lock per notebook.

How verified. The next queue run hit the session limit three times, and the guard stopped each within 0.1 minutes with exit code 1 (Table 11, `driver4.log` rows). Unit tests cover a genuine run, a refused push, output from another version, stale-file clearing, and kernel code that does not write the `run_id`. The two invalid outputs were moved to `data/eval/runs/_invalid/`, and the three affected timing records were marked invalid in place.

Table 11: The stale-output event (`driver3.log`) and the first guarded rerun (`driver4.log`), parsed per queue entry. Rows are the entries that hit a refused push or a missing result. "run_id issued" is whether the launcher already generated a run identifier.

log	queue entry	run_id issued	push	guard fired	recorded result
driver3	filbench, Qwen3.5-4B-Base	no	accepted	no	read COMPLETE, exit 0, 1.7 min
driver3	ph-eval, Qwen3.5-4B	no	refused	no	exit 1, 0.1 min
driver3	filbench, gemma-4-E4B	no	refused	no	read COMPLETE, exit 0, 0.1 min
driver4	filbench, Sailor2-8B	yes	refused	yes	exit 1, 0.1 min
driver4	filbench, Qwen3.5-4B-Base	yes	refused	yes	exit 1, 0.1 min
driver4	ph-eval, Qwen3.5-4B	yes	refused	yes	exit 1, 0.1 min

Effect on results. None on reported numbers. The two mislabelled records were caught the same hour and never entered a comparison. Every reported run carries a `run_id` that matches its push (Table 4).

2.7 Filesystem and warm-container provenance

These are two separate defects, plus one operational weakness. None changes a score, but each could attach one run's files to another.

(a) Windows turns `:` into an alternate data stream.

- *Symptom.* After download, four early Kaggle FilBench folders contain a 0-byte file named `details_global_mmlu_all_tgl_mcf`. The run's results summary still showed Global-MMLU scores, and nothing reported an error.
- *Root cause.* lighteval names Global-MMLU details `details_global_mmlu_all_tgl_mcf:<subject>|0_<timestamp>.parquet`. Windows cannot create `|` (the earlier symptom was an `[Errno 22]` abort, METHODS-001 3.5). On NTFS, `name:stream` addresses an alternate data stream, so after the `|` rename each subject's parquet went into a hidden stream of one empty host file. Enumerating the streams with `FindFirstStreamW` shows 57 streams, one per subject, holding about 3 MiB per folder (Table 12).
- *Fix.* Rename `|` → `__` and `:` → `--` inside the container before download, and apply the same mapping again in the local downloader. The scorer refuses a run with any 0-byte details file or with zero Global-MMLU items.
- *How verified.* The three full A100 runs each arrived with 57 Global-MMLU files named `...mcf--<subject>__0_...`, and no 0-byte details file exists in any A100 folder.
- *Effect on results.* None. The affected folders are sample runs and failed full runs, which are not used for reported numbers.

Table 12: The 0-byte file `details_global_mmlu_all_tgl_mcf` in each affected folder, and the NTFS alternate data streams behind it, enumerated with the Win32 stream API.

Kaggle run folder	lighteval timestamp	size (bytes)	streams	hidden (MiB)
google-gemma-4-e4b	2026-09-29 11-16	0	57	2.9
qwen-qwen3-5-4b-base	2026-09-30 05-06	0	57	3.0
qwen-qwen3-5-9b-base	2026-09-29 11-54	0	57	3.0
sail-sailor2-8b	2026-09-29 11-00	0	57	3.0

A related leftover: the Kaggle launcher clears stale outputs with non-recursive patterns (`results_*.json`, `run_meta.json`, `*.log`, `predictions.jsonl`). A lighteval `results/` subtree from an earlier run therefore survives next to the `run_meta.json` of a later, failed run. Two folders are in this state today (Table 13). They are not used, and the scorer would refuse both because of their 0-byte files. The general gap is that nothing binds a details timestamp to a `run_id` (Section 5).

Table 13: Kaggle FilBench folders: when the run named in `run_meta.json` was pushed (decoded from its `run_id`), whether it completed, and which lighteval details timestamps sit in the folder.

Kaggle run folder	run pushed (UTC)	that run	details timestamps	details older?
google-gemma-4-12b	09-30 05:27Z	failed	none	no
qwen-qwen3-5-9b-base	09-30 05:19Z	failed	09-29 11-54	yes
sail-sailor2-8b	09-30 05:10Z	failed	09-29 11-00	yes
qwen-qwen3-5-4b-base	09-30 04:38Z	complete	09-30 05-06	no
google-gemma-4-e4b	09-29 11:02Z	complete	09-29 11-16	no

(b) A warm Modal container carries files across calls.

- *Symptom.* The Gemma 4 12B folder of the full A100 runs came back holding Sailor2-8B's and Qwen3.5-9B-Base's details, and the Qwen3.5 folder held Sailor2's.
- *Root cause.* Modal reused one warm container for the three sequential calls, and the output directory `/out` persisted between them. lighteval's `start_time`/`end_time` fields come from a monotonic clock, and they run continuously across the three calls, from 16 s to 4,512 s. A fresh container would restart that clock (Table 14).
- *Fix.* Delete `/out` at the start of every call. The foreign files were moved to `_stray-warm-container/`, not deleted. The scorer reads only details under the run's own model path, outside `_stray`, and requires exactly one lighteval timestamp.
- *How verified.* Each model folder now resolves to its own 69 details files from a single timestamp (Table 14).
- *Effect on results.* None on scores, since the scorer filters by model. Without the filter, a naive glob over the Gemma folder would have scored 3 × 69 files as one model.

Table 14: The three full FilBench calls on one A100 80GB PCIe, in call order. "Own files" and "foreign files" count details parquets of the folder's model and of other models (moved aside). Eval start/end are lighteval's `config_general` times in seconds; they continue across calls because one container served all three.

model	own files	foreign files	eval start (s)	eval end (s)	eval time (s)
Sailor2-8B	69	0	16	889	873
Qwen3.5-9B-Base	69	69	920	2,930	2,011
gemma-4-12B	69	138	2,963	4,512	1,549

(c) Fragile local drivers. The drivers that push and poll Kaggle runs execute on a laptop, and twice a driver stopped while its kernel kept running:

- The FULL driver's log stops mid-poll during Qwen3.5-4B. The shell was killed when the host ran low on memory.
- The Gemma 4 E4B-it driver exited on a dropped connection after 26.5 minutes.

Both kernels completed on Kaggle. Their outputs were fetched by hand, and their `run_id`s match the ones logged at push (Table 4). Two lessons: run one local polling driver at a time, and never make GPU work depend on the laptop staying up.

3. Guards now in the runner

Figure 2 shows where each guard sits in the run pipeline, and the list below gives the code location for each.

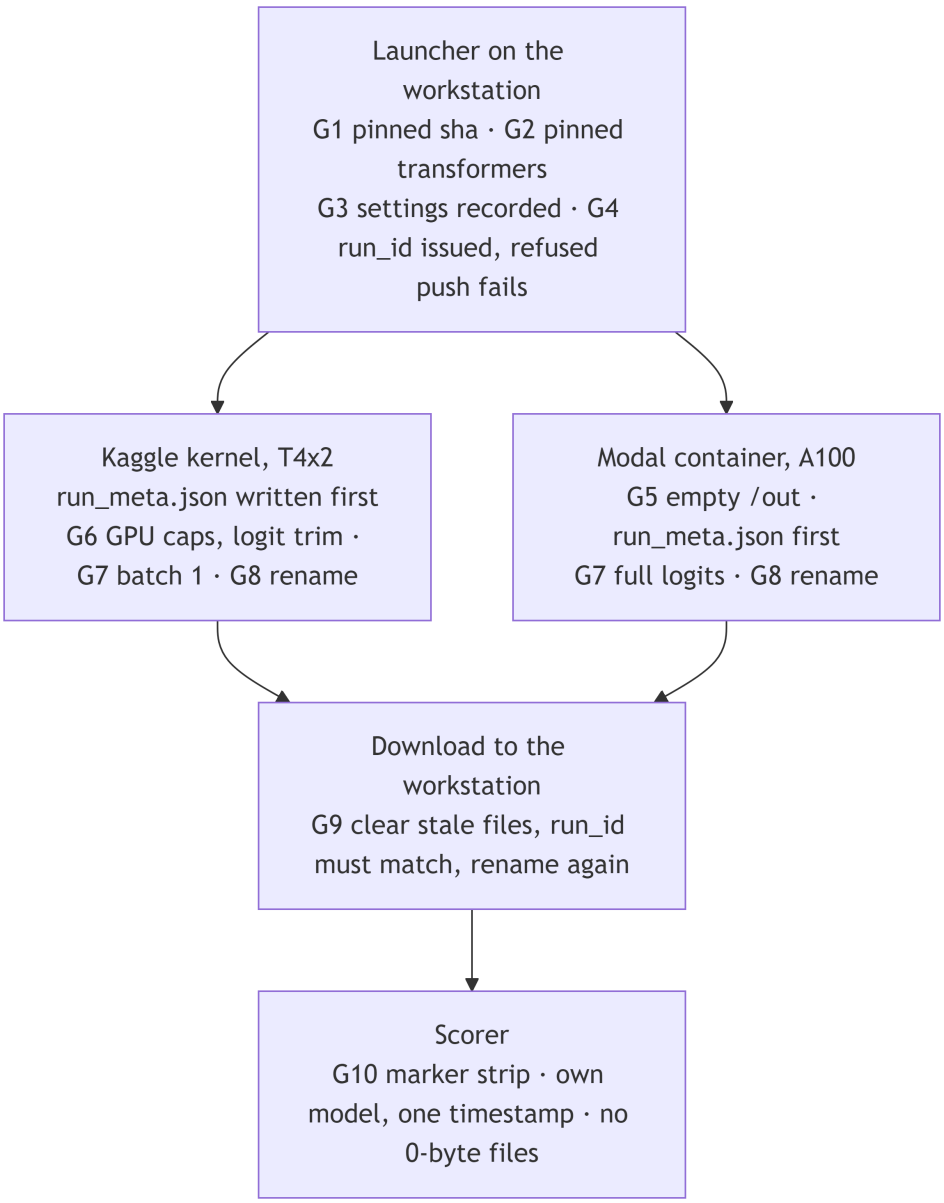


Figure 2: Run pipeline for PH-EVAL generation and FilBench MCF, with the guards (G#) at each stage.

- **G1 Pinned revisions.** `resolve_revision()` expands the `configs/models.yaml` pin to a full SHA. It is passed to every `from_pretrained` and recorded (`src/phfm/eval/kaggle.py`).
- **G2 Pinned transformers.** `TRANSFORMERS_VERSION = "5.17.0"` is installed after lighteval in every kernel and image (`kaggle.py`, `scripts/eval/kaggle_filbench.py`, `scripts/inference/modal_filbench.py`). The imported version is recorded.
- **G3 Template settings recorded.** `enable_thinking` defaults to False, and the MCF chat template defaults to off. Both are written to `run_meta.json` (`scripts/eval/ph_eval_generate.py`, `scripts/eval/filbench_run.py`; METHODS-001 for the template).
- **G4 Push and run identity.** A refused push raises, and the kernel must write the launcher's `run_id` (`run_kernel()` refuses code without the placeholder). The queue drivers stop on a failure under one minute (`scripts/inference/m1_*.py`).
- **G5 Clean container output.** `shutil.rmtree("/out")` runs at the start of each Modal call.
- **G6 Memory.** The `max_memory` caps are logged per run (`--gpu0-gib`, `--gpu1-gib`). The `keep_last_logits` proxy raises instead of returning a wrong log-probability, and `PYTORCH_ALLOC_CONF=expandable_segments:True` is set (`src/phfm/eval/lighteval_patches.py`).
- **G7 Batch and platform per memory budget.** The full-run T4 driver uses batch 1 with 11/12 GiB for the 12B. The A100 driver uses batch 4 with full logits.

- **G8/G9 File names and download.** The rename runs in the container and again locally. Stale files are cleared, and a `run_id` mismatch fails the download.
- **G10 Scorer.**
  - It strips the thought marker from every model alike.
  - It reads only the run's own model details from exactly one timestamp.
  - It refuses 0-byte details and a missing Global-MMLU.
  - It raises on duplicate item ids (`src/phfm/eval/reports/scorecard.py`, `scripts/eval/score_part_c.py`).

4. Reproducibility

- **Pins.** The revisions are in Table 2, and transformers 5.17.0 is recorded per run. lighteval is installed from git main and reports version 0.13.1.dev0. Its commit is **not** recorded: `lighteval_sha` in every results file is `?` (Section 5).
- **Commands.**
  - PH-EVAL FULL on Kaggle: `python scripts/inference/m1_full_ph_eval.py [--only <repo>]`, which wraps `scripts/eval/kaggle_run.py`.
  - Full FilBench on T4: `python scripts/inference/m1_part_c.py`.
  - Full FilBench on A100: `uv run --with modal==1.6.0 --with pyyaml modal run scripts/inference/modal_filbench.py --stage partc`. The harness recheck of METHODS-001 is `--stage recheck`.
- **Host portability.** The kernels are thin launchers. The method lives in host-agnostic payloads (`scripts/eval/ph_eval_generate.py`, `scripts/eval/filbench_run.py`), so the same code ran on Kaggle and Modal.
- **Outputs and tables.** Outputs go to `data/eval/runs/<slug>/`: `predictions.jsonl` and `run_meta.json` for generation, and `filbench/` or `filbench-modal/` for MCF. Every table and figure in this note is regenerated by `python experiments/reports/_eng002.py`, which also asserts the counts quoted in the text.
- **Timing, for planning only** (Table 15). These are single runs on shared infrastructure. The A100 variant was PCIe, so the times are not comparable with SXM4 measurements elsewhere.

Table 15: Measured times. T4x2 = Kaggle `NvidiaTeslaT4`; A100 = A100 80GB PCIe. PH-EVAL FULL (175 prompts, 512 new tokens): launcher wall time from push to download (driver `timings.jsonl`; the Qwen3.5-4B and Gemma 4 E4B-it runs were fetched by hand after their drivers stopped, so they have no clean wall time). FilBench: lighteval's own evaluation time, excluding model download and load.

model	workload	host	time
gemma-4-12B-it	PH-EVAL FULL	T4x2	90.8 min
Qwen3.5-9B	PH-EVAL FULL	T4x2	83.7 min
Sailor2-8B-Chat	PH-EVAL FULL	T4x2	60.7 min
Sailor2-8B	FilBench, all items	A100	14.5 min
Qwen3.5-9B-Base	FilBench, all items	A100	33.5 min
gemma-4-12B	FilBench, all items	A100	25.8 min

5. Limitations and open issues

- **Open: EVAL-001 still shows the old score (2.4).** The scorer is fixed and the scorecard regenerated, but EVAL-001 still reports Gemma 4 12B-it at 87.1% on the auto-scored subset. It should be updated to 90.0% (63/70). Other models are unchanged.
- **Open: run identity for lighteval details.** `run_id` binds `run_meta.json` to a push, but not the details files. On Kaggle, a stale `results/` subtree can sit next to a newer failed run (Table 13). Clearing the output folder recursively, or writing `run_id` into the details path, would close this.
- **Open: per-row logit window.** The `keep_last_logits` window is sized per batch. The fix for mixed-length batches on T4 is batch 1, and it has not yet been exercised on a full run.
- **fp16 only.** The fp16 check (2.2) rules out overflow-level failure, not small accuracy shifts. The public leaderboard lists bf16, and no bf16 twin run exists on the A100.
- **Unrecorded environment.** The GPU image's torch and CUDA versions are not in `run_meta.json`. Only the CPU probe recorded torch (2.10.0+cpu). The lighteval commit is unrecorded, as noted in Section 4.
- **Estimates and lost logs.** The memory figures in Table 8 are arithmetic. The row-position figures in Table 10 assume fp16 `[rows, keep, V]` tensors. Two numbers come from the team's run notes, not re-derivable files, because their kernel logs were overwritten by later runs in the same folder: the first 1.57 GiB request (quoted from METHODS-001) and the 14.43 GiB allocation on GPU 1.

- **Single runs.** Each configuration ran once. The throughput numbers are indicative only.
- **Scope.** Only the inference layer is covered. The FilBench harness (chat template for MCF, lighteval's gather-slice bug, dataset loading) is in METHODS-001. Model comparisons are in EVAL-001 (PH-EVAL) and RESULT-002 (FilBench).

Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

AI assistance. Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.

References

[1] L. J. V. Miranda and others, "FilBench: Can LLMs Understand and Generate Filipino?," *arXiv preprint arXiv:2508.03523*, 2025.